RESEARCH KNOWLEDGE REPOSITORY

WITH SEMANTIC SEARCH

Project-Based Learning Report

Submitted in partial fulfillment of the requirements for the award of the degree
Bachelor of Technology
Course: Database Engineering and Software Development (DBE & DSD) – 25CS1302E

Team Member	Roll ID
GUDUPALLY AKSHAYA REDDY	2510080024
NIHARIKA	2510080022
HARSHINI	2510080019
VARNA	2510080023

Under the guidance of

Mrs. Melinda
Koneru Lakshmaiah Education Foundation (KLEF)
Aziz Nagar – 500075

2026



ABSTRACT

The Research Knowledge Repository with Semantic Search is a web-based academic system designed to provide a centralized platform for storing, managing and discovering research papers. Traditional keyword-based retrieval depends strongly on matching words between a user query and a document. This can cause relevant papers to be missed when the same concept is expressed using different terminology.

The proposed system addresses this limitation by converting extracted research-paper text into semantic vector embeddings and comparing a user query embedding with the stored document representations. The working prototype uses a Flask-based backend, PDF processing, a Sentence Transformer embedding model and a web frontend. The implemented pipeline covers authentication, PDF upload, text extraction, embedding generation, storage, semantic search, result ranking and repository-management operations.

The repository is not limited to search. Users can view stored papers, download documents, replace an existing paper through the update operation, and delete papers. The application therefore combines information retrieval with practical document-management functions in one academic prototype.

The project progressed through review stages from problem identification and proposed solution to literature-oriented methodology and implementation. The documented working workflow demonstrates successful PDF processing, generation of 384-dimensional embeddings in the tested configuration, semantic-search execution and repository operations. The report emphasizes functional evidence and identifies quantitative retrieval evaluation as a next step.

Keywords: Research Repository, Semantic Search, Sentence Transformers, Embeddings, PDF Text Extraction, Similarity Search, Flask, Information Retrieval.

LIST OF FIGURES

Figure	Description
1	Logical system architecture
2	End-to-end document processing workflow
3	Semantic-search data flow
4	Embedding and similarity pipeline
5	Authentication flow
6	Frontend home/upload interface
7	Repository page
8	Semantic-search results
9	Backend processing console
10	Update-paper interface
11	Paper-details page
12	Repository management flow
13	Functional evaluation workflow
14	Screenshot evidence – login
15	Screenshot evidence – upload
16	Screenshot evidence – repository
17	Screenshot evidence – search
18	Screenshot evidence – update
19	Screenshot evidence – paper details
20	Screenshot evidence – download/delete

LIST OF TABLES

Table	Description
1	Project technology stack
2	Functional requirements
3	Non-functional requirements
4	User and access responsibilities
5	Core repository data entities
6	Document-processing stages
7	Module decomposition
8	API/route summary
9	Repository operations
10	Semantic-search evaluation plan
11	Functional test cases
12	Observed intermediate results
13	Limitations and mitigation
14	Future enhancement roadmap

TABLE OF CONTENTS

1. INTRODUCTION	7.2 Upload Interface
1.1 Background and Context	7.3 Search Interface
1.2 Problem Statement	7.4 Repository Interface
1.3 Objectives	7.5 Paper Details and Update Pages
1.4 Scope	8. BACKEND IMPLEMENTATION
1.5 Significance of the Project	8.1 Flask Application
1.6 Academic relevance	
2. REQUIREMENT ANALYSIS	8.2 Authentication Services
2.1 Existing System	8.3 PDF Processing
2.2 Proposed System	8.4 Embedding Service
2.3 Functional Requirements	8.5 Repository Operations
2.4 Non Functional Requirements	9. EXPERIMENTS AND TESTING
2.5 User and Access Requirements	9.1 Functional Testing
3. METHODOLOGY	9.2 Semantic Search Testing
3.1 Development Approach	9.3 Repository Testing
3.2 Overall Workflow	9.4 Security and Access Testing
3.3 Document Processing	10. RESULTS AND DISCUSSION
3.4 Semantic Retrieval Principle	10.1 Observed Results
3.5 Module Decomposition	10.2 Evaluation Plan
4. SYSTEM DESIGN	10.3 Practical Contribution
4.1 Architecture	10.4 Adaptability
4.2 Data Flow	11. CURRENT STATUS, LIMITATIONS AND FUTURE WORK
4.3 Application Routes	12. CONCLUSION
4.4 Validation and Security	REFERENCES
5. DATABASE AND STORAGE DESIGN	SCREENSHOT DOCUMENTATION GUIDE
5.1 Repository Data Model	
5.2 Document Metadata	
5.3 User Isolation	
5.4 Vector Representation	
6. SEMANTIC SEARCH DESIGN	
6.1 Embedding Generation	
6.2 Query Processing	
6.3 Similarity Calculation	
6.4 Result Ranking	
7. FRONTEND IMPLEMENTATION	

1. INTRODUCTION

1.1 Background and Context

Research papers are essential for academic learning, project development and scientific progress. As the volume of available research increases, the task of locating useful documents becomes more time-consuming. A repository therefore needs to support not only storage but also efficient discovery.

Conventional keyword search is useful when the query and document use the same or closely matching terms. However, a researcher may describe a concept using a different expression from the wording used inside a paper. A semantic retrieval approach attempts to compare meaning-oriented representations rather than relying only on exact word overlap.

The Research Knowledge Repository combines document management with semantic retrieval. A research-paper PDF enters through the upload interface, text is extracted, the extracted content is transformed into an embedding, and the representation is stored with the document. During retrieval, the user's natural-language query is embedded using the same model and compared with stored document embeddings.

1.1 Project Context

The project was developed as a practical academic prototype. Its emphasis is on connecting frontend interaction, backend processing, document storage and semantic retrieval into one working workflow. The project documentation records progression across Review 1, Review 2 and Review 3, moving from problem definition and objectives toward methodology and implementation.

1. INTRODUCTION – PROBLEM, OBJECTIVES AND SCOPE

1.2 Problem Statement

Traditional keyword-based retrieval may return irrelevant documents or fail to retrieve semantically related research papers when terminology differs. At the same time, researchers need a single location where papers can be uploaded, stored, viewed, searched, downloaded, updated and deleted. The project therefore addresses both research discovery and repository-management requirements.

1.3 Objectives

- Develop a centralized repository for research documents.
- Provide authenticated access to repository operations.
- Allow users to upload and manage research-paper PDFs.
- Extract readable text from uploaded PDF documents.
- Generate semantic embeddings from extracted research content.
- Store documents together with metadata and semantic representations.
- Support natural-language semantic search.
- Rank and display relevant research papers.
- Provide view, download, update and delete operations.
- Create a practical foundation for future intelligent research-support features.

1.4 Scope

The current scope covers authentication, research-paper PDF upload, text extraction, embedding generation, storage, natural-language semantic search, ranked retrieval, repository viewing, paper details, downloading, updating and deletion. Advanced capabilities such as summarization, question answering and recommendation are identified as future extensions rather than claimed as completed functionality.

1. INTRODUCTION – SIGNIFICANCE

1.5 Significance of the Project

The significance of the project lies in practical integration. Instead of demonstrating PDF processing, machine learning and web development as isolated experiments, the system connects them through a single user-facing workflow. A paper uploaded by a user becomes searchable through its semantic representation and remains manageable through repository operations.

The project is also useful from a software-engineering perspective because each stage has a clear responsibility. The frontend collects input and presents results; the Flask backend manages requests and application logic; PDF processing produces searchable text; the embedding model creates semantic representations; and the repository layer stores the information required for later retrieval.

1.6 Academic Relevance

The project demonstrates concepts from web development, database engineering, information retrieval and machine learning. It also provides a clear basis for a Project Based Learning demonstration because an evaluator can observe a document moving from upload through processing to search and repository management.

1.7 Practical Contribution

The project does not claim that semantic search itself is a new research concept. Its practical contribution is the integration of research-paper management, PDF processing, semantic embeddings, natural-language retrieval and repository operations in one academic prototype. This framing is consistent with the documented review progression and avoids overstating novelty.

2. REQUIREMENT ANALYSIS

2.1 Existing System

A basic research repository may provide file storage and keyword search. Such a design can become difficult to use as the number of documents increases, especially when researchers remember a concept but not the exact terminology used in the paper.

A purely file-based workflow also separates document management from intelligent retrieval. The proposed project addresses this by treating the research paper and its semantic representation as connected repository data. The backend coordinates upload, processing, storage and search.

2.2 Proposed System

The proposed platform provides authenticated access, research-paper upload, PDF text extraction, embedding generation, repository storage, semantic search and management operations. The same application therefore supports both discovery and lifecycle management.

Area	Existing limitation	Proposed capability
Storage	Files may be difficult to organize	Centralized repository
Search	Exact-word dependence	Semantic similarity retrieval
Processing	Manual or separate processing	Automatic PDF extraction and embedding
Management	Basic file handling	View, download, update and delete
Access	May not be user-aware	Authenticated application workflow

2. REQUIREMENT ANALYSIS – FUNCTIONAL REQUIREMENTS

2.3 Functional Requirements

ID	Requirement	Priority
FR-01	User registration and login	High
FR-02	Authenticated repository access	High
FR-03	Research-paper PDF upload	High
FR-04	PDF text extraction	High
FR-05	Semantic embedding generation	High
FR-06	Repository storage	High
FR-07	Natural-language semantic search	High
FR-08	Similarity-based result ranking	High
FR-09	Paper view/details	Medium
FR-10	PDF download	Medium
FR-11	Paper update with reprocessing	High
FR-12	Paper deletion	High

2.4 Functional Workflow

1. A user authenticates through the login workflow.
2. The user uploads a research-paper PDF.
3. The backend validates the file and extracts readable text.
4. The extracted text is converted into a semantic embedding.

5. The paper metadata and semantic representation are stored.
6. The user submits a natural-language query.
7. The same embedding model creates a query representation.
8. The system compares the query representation with stored document representations.
9. Results are ranked and displayed.
10. The repository allows subsequent view, download, update and delete operations.

3. REQUIREMENT ANALYSIS – NON-FUNCTIONAL REQUIREMENTS

2.5 Non-Functional Requirements

Requirement	Expected behavior
Usability	Clear pages for login, upload, search and repository management.
Maintainability	Frontend, backend, processing and retrieval responsibilities remain separable.
Reliability	Invalid files and missing resources should be handled without corrupting repository state.
Security	Protected operations require authentication and passwords are handled through hashing.
Performance	Search should return a practical ranked result set for the repository size.
Scalability	Architecture should allow larger repositories and improved vector indexing later.
Traceability	Uploaded documents can be followed through extraction, embedding and storage.
Extensibility	Future summarization, question answering and recommendation can build on the repository.

3.4 Validation Requirements

1. Reject unsupported upload formats such as non-PDF files.
2. Require authenticated access to protected repository functions.
3. Validate required form inputs.
4. Check that a requested paper exists before view, update, download or delete operations.
5. Regenerate extracted text and embeddings when a paper is replaced.
6. Use the same embedding model for documents and search queries.

2. REQUIREMENT ANALYSIS – USER AND ACCESS

2.7 User Responsibilities

The documented prototype uses authentication to establish an authenticated application session. The authenticated user can access protected repository functions. The project design treats account isolation as an important requirement for a multi-user repository: a user's repository operations should be scoped to that user's documents.

User action	System responsibility
Register	Validate inputs, check username availability and create an account.
Login	Verify credentials and establish an authenticated session.
Upload	Accept PDF, extract text, generate embedding and store paper data.
Search	Generate query embedding and retrieve ranked papers.
View	Return paper details for an accessible document.
Download	Serve the stored PDF for an accessible document.
Update	Replace PDF, regenerate text/embedding and update stored data.
Delete	Remove the selected accessible paper.
Logout	End the authenticated session.

2.8 Account Isolation

For a multi-account implementation, repository records must be associated with their owning user and repository/search/view/update/download/delete queries

must enforce that association. This is particularly important because the project is intended to serve multiple authenticated users. The documented development work identified account-specific visibility as a necessary implementation concern.

3. METHODOLOGY

3.1 Development Approach

The project follows an iterative development approach. Review 1 establishes the problem, objectives, feasibility and core modules. Review 2 strengthens the information-retrieval direction through literature-oriented study and identifies semantic retrieval as the central intelligent capability. Review 3 concentrates on implementation, working modules and intermediate results.

3.2 Methodology Principles

- Keep document processing separate from user-interface concerns.
- Use one embedding model consistently for document and query representations.
- Persist the document information needed for repository management and later retrieval.
- Provide explicit operations for view, download, update and delete.
- Evaluate current functionality through observable application behavior.
- Distinguish functional evidence from quantitative search-quality claims.
-

3.3 End-to-End Method

The complete method is document-to-representation-to-retrieval. A PDF becomes readable text, the text becomes a semantic vector, and the vector becomes searchable repository information. A query follows the same representation process so that the query and stored documents can be compared in the same embedding space.

3. METHODOLOGY – OVERALL WORKFLOW

3.4 Overall Workflow

10. User authentication establishes an authenticated session.
11. User selects a research-paper PDF.
12. Backend validates the uploaded file.
13. PDF pages are read and text is extracted.
14. Extracted text is prepared for semantic representation.
15. Sentence Transformer generates a document embedding.
16. Paper metadata and embedding are stored.
17. User enters a natural-language research query.
18. The same model generates a query embedding.
19. Similarity is calculated between query and stored document embeddings.
20. Results are ranked according to similarity/relevance.
21. User can open, download, update or delete a repository paper.

UPLOAD → EXTRACT → EMBED → STORE → SEARCH → RANK → RETRIEVE

The workflow is intentionally linear for the core processing path. Update operations return to the extraction and embedding stages so that the stored semantic representation remains aligned with the replacement document.

3.4 Document Processing

Input	Processing	Output
Research-paper PDF	File validation	Accepted PDF
PDF	Page-by-page text extraction	Readable research text
Extracted text	Semantic embedding model	384-dimensional embedding in tested configuration
Paper + metadata	Repository storage	Searchable document record
Natural-language query	Same embedding model	Query embedding
Query + stored embeddings	Similarity comparison	Ranked research-paper results

3.6 Processing Characteristics

The prototype uses PDF text extraction to turn uploaded documents into machine-readable content. The extracted text is then passed to a Sentence Transformer model. The documented implementation generated 384-dimensional representations in the tested workflow. The use of the same model for documents and queries is essential because the representations must be comparable.

3.7 Processing Integrity

When a document is updated, the replacement PDF is processed again. The system therefore does not retain the old extracted text or embedding as the representation of the new file. This keeps the repository representation synchronized with the current document.

3.8 Semantic Retrieval Principle

A semantic embedding is a numerical representation intended to capture information about the meaning of text. Instead of asking whether a query contains exactly the same words as a paper, the system compares the vector representation of the query with the vector representation of stored research content.

For example, a user may ask a question using one terminology while a paper discusses the same concept using another. Semantic representation can make such relationships retrievable when the underlying model places the related texts near one another in embedding space.

3.9 Similarity Concept

The working retrieval query uses vector distance to compare a query embedding with stored embeddings and converts the distance relationship into a similarity score. The system then orders candidate papers by relevance. The exact ranking behavior should be interpreted as retrieval behavior rather than as proof of scientific relevance.

3.10 Why Semantic Search

- Reduces dependence on exact query wording.
- Supports natural-language research questions.
- Provides a common representation for queries and documents.
- Allows relevance-oriented ranking instead of simple keyword presence.
- Creates a foundation for future question answering and recommendation.

4. SYSTEM DESIGN – ARCHITECTURE

4.1 Layered Architecture

The working prototype is organized into a frontend layer, a Flask backend and a data/AI layer. The frontend handles user interaction. The backend manages authentication, requests, file processing, PDF extraction, repository operations and semantic retrieval. The data/AI layer stores research information and supports semantic representations.

Layer	Responsibilities
Frontend	Login, upload, search, repository display, paper details and management controls.
Flask Backend	Routing, authentication, request handling, file processing and application logic.
PDF Processing	Read uploaded PDF pages and extract searchable text.
Embedding / AI	Generate document and query embeddings using Sentence Transformer.
Data Storage	Store papers, metadata and associated representations.
Retrieval	Compare query/document representations and rank results.

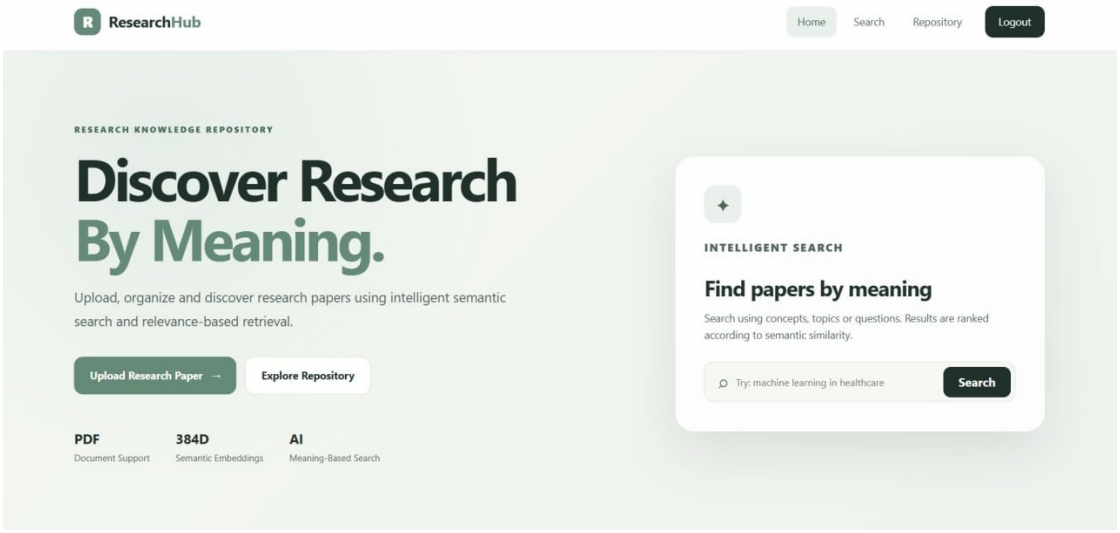


Figure 1. Working-project interface/architecture evidence from the original documentation.

4. SYSTEM DESIGN – DATA FLOW

4.2 Data Flow

The primary data flow begins at the upload interface and ends at searchable repository content. A user submits a PDF, the backend extracts text, the embedding component creates a vector, and the repository stores the paper information. Search reverses the direction: the user supplies a query, the system creates a query embedding, compares it with stored representations and returns ranked results.

Stage	Data	Result
Input	PDF file	Validated upload
Extraction	PDF pages	Research text
Representation	Research text	Document embedding
Persistence	Paper + metadata + embedding	Repository record
Query	Natural-language text	Query embedding
Comparison	Query + document vectors	Similarity values
Retrieval	Ranked candidates	Search results

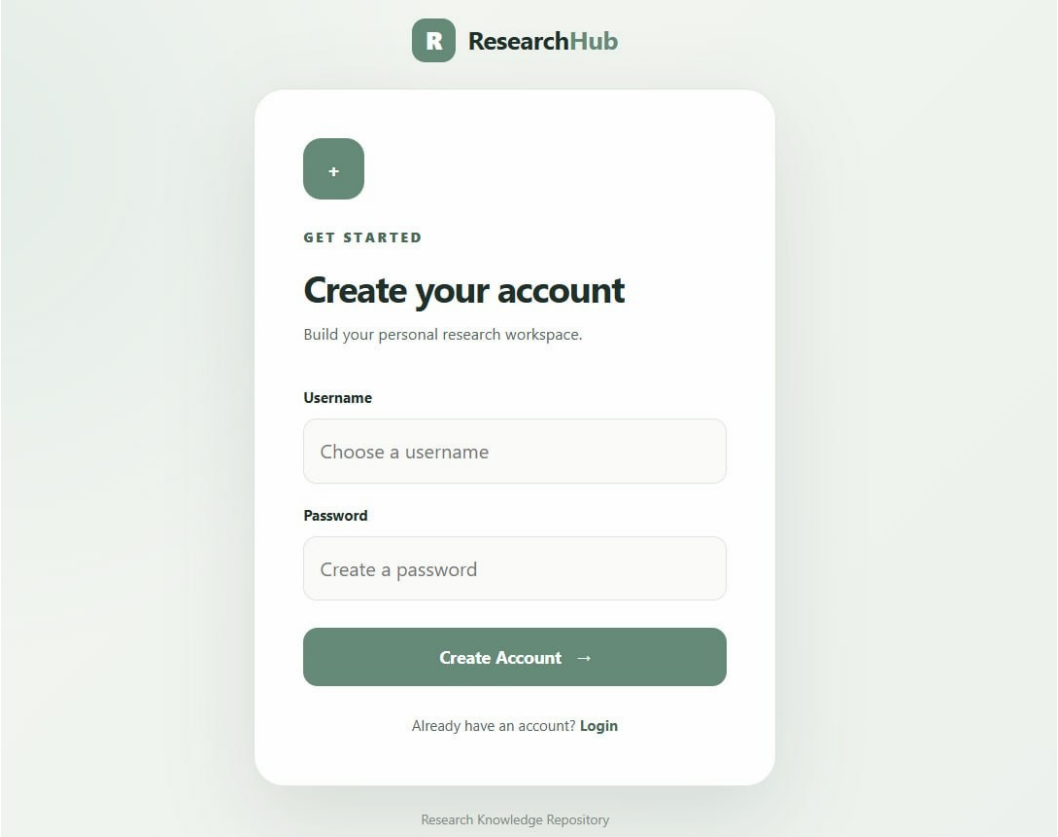


Figure 2. Repository interface evidence supporting the documented data-flow stages.

4.3 Application Route Design

Route/Function	Purpose	Access
/register	Create a new user account	Public
/login	Authenticate user	Public
/logout	End authenticated session	Authenticated
/	Upload and process a research PDF	Authenticated
/search	Process a semantic-search query	Authenticated
/repository	Display stored papers	Authenticated
/paper/<id>	Display paper details	Authenticated
/download/<id>	Download stored PDF	Authenticated
/update/<id>	Replace and reprocess a paper	Authenticated
Delete operation	Remove selected paper	Authenticated

The route structure reflects the separation between authentication, document processing, search and repository management. Protected routes require an authenticated session so that repository actions are not exposed to unauthenticated users.

4.4 Request Lifecycle

22. Browser sends request.
23. Flask route validates authentication and input.
24. Backend performs processing or database operation.
25. Result is rendered or returned to the frontend.
26. Repository state is updated when the operation changes stored data.

4.5 Security and Validation

The authentication workflow uses Flask-Login and password hashing. Registration validates required fields, checks username uniqueness and stores a generated password hash rather than a plaintext password. Login verifies the supplied password against the stored hash and establishes an authenticated session.

Control	Purpose
Password hashing	Avoid storing raw passwords.
Authenticated routes	Restrict repository operations to logged-in users.
File-type validation	Accept research-paper PDF input only.
Required-field validation	Prevent incomplete requests.
Resource existence checks	Avoid operations on missing papers.
User ownership filtering	Prevent cross-account repository access.
Controlled file storage	Keep uploaded documents in the configured upload area.

4.6 Security Consideration for Multi-User Operation

Authentication alone does not guarantee data isolation. Repository queries must also associate documents with the current user. The account-isolation requirement should therefore be enforced at the database query level for listing, searching, viewing, downloading, updating and deleting papers.

5.1 Repository Data Model

The repository stores information required to identify, display, retrieve and manage research papers. The documented working implementation stores paper metadata including title, filename, extracted text and embedding, with repository operations also using identifiers and upload timestamps.

Data element	Purpose
Paper ID	Unique identifier for a repository paper.
Title	Human-readable research-paper title derived from the uploaded filename in the documented workflow.
Filename	Stored PDF filename used for file access.
Extracted text	Searchable textual representation of the PDF content.
Embedding	Semantic vector representation used for similarity search.
Uploaded timestamp	Supports repository ordering and management history.
User/account association	Required for isolated multi-user repositories.

5.2 Storage Principle

The repository record connects the physical PDF with the extracted textual representation and the semantic vector. This means a search result can lead back to a manageable document rather than producing only a score.

5.3 Logical Relationship

The multi-user design is best represented by a one-to-many relationship between a user and research papers: one authenticated user can own multiple uploaded papers, while each paper belongs to one user. This relationship enables account-specific repository views and searches.

Entity	Relationship	Purpose
User	1 → many Papers	Separates documents by account.
Paper	1 → 1 stored PDF	Connects metadata to the physical file.
Paper	1 → 1 extracted text	Keeps searchable content with its source.
Paper	1 → 1 embedding	Enables semantic retrieval.

5.4 Account-Aware Queries

Repository listing should filter records by the authenticated user's identifier. Semantic search should apply the same account restriction before ranking results. Paper detail, download, update and delete operations should verify that the requested paper belongs to the current user before performing the operation.

This design directly addresses the multi-account requirement: creating a new account should begin with an empty repository, while logging into an existing account should display only that account's papers.

5. DATABASE AND STORAGE DESIGN – NORMALIZATION

5.5 Normalization Considerations

The repository data model separates user identity from paper records rather than repeating user information inside every paper. A normalized design stores each user's identity once and references that identity from paper records. Paper metadata and semantic representation remain associated with the document entity.

Design concern	Normalized approach	Benefit
User information	Stored in user entity	Avoids repeated identity data.
Paper metadata	Stored once per paper	Simplifies updates.
PDF content	Stored/referenced through paper record	Keeps file management explicit.
Embedding	Stored as document representation	Supports vector retrieval.
Ownership	User ID associated with paper	Enables account isolation.

5.6 Update Consistency

When a paper is replaced, title/filename metadata, extracted text and embedding must be updated together. Otherwise the repository could display one document while semantic search still uses the representation of an older file. The documented update workflow regenerates the extracted text and embedding after a replacement PDF is supplied.

VECTOR REPRESENTATION

5.7 Embedding Storage

The semantic component produces a fixed-length vector representation for each processed research document. The tested implementation records a 384-dimensional embedding. The same representation size is used for query embeddings produced by the same model.

5.8 Vector Similarity Retrieval

During search, the backend creates an embedding for the user's query and compares it against stored document embeddings. The retrieval query orders candidate records using vector distance. A similarity score can then be presented as a relevance-oriented value for the returned papers.

5.9 Storage Requirements

- Document records must remain linked to their source PDFs.
- Embeddings must remain aligned with extracted text.
- Query and document embeddings must use the same model configuration.
- Vector operations should be indexed or optimized as repository size grows.
- Account filtering should be applied before returning results to the user.

The current prototype demonstrates the semantic workflow. For a larger repository, a dedicated vector index or optimized vector-database strategy can be evaluated as part of future scalability work.

5.10 Repository Operations

Operation	Input	Processing	Output
Create/Upload	PDF	Validate → extract → embed → store	New repository paper
View	Paper ID	Verify access → read metadata/text	Paper details
Download	Paper ID	Verify access → locate PDF	PDF response
Update	Paper ID + replacement PDF	Verify access → extract → embed → update	Updated paper
Delete	Paper ID	Verify access → remove record/file as designed	Repository state updated
List	Authenticated user	Filter by owner → order records	User-specific repository
Search	Query text	Embed query → compare → rank	User-specific ranked results

Repository management is an important part of the system because semantic search is more useful when every retrieved document can be inspected and managed. The operations also provide visible evidence of full-stack integration.

6. SEMANTIC SEARCH DESIGN

6.1 Embedding Generation

The working prototype uses Sentence Transformers to generate semantic embeddings. The documented implementation uses the all-MiniLM-L6-v2 model configuration and produces 384-dimensional representations in the tested workflow.

6.2 Document Embedding

27. Read the uploaded PDF.
28. Extract text from all available pages.
29. Prepare the extracted text for the embedding model.
30. Pass the text to the Sentence Transformer.
31. Receive the fixed-length vector representation.
32. Store the embedding with the document information.

6.3 Query Embedding

The search query follows the same representation path. A natural-language query is passed to the same embedding model, producing a vector in the same semantic space as the stored documents.

6.4 Query Processing

The search interface accepts a natural-language research question or topic. The backend converts that query into an embedding rather than treating it only as a sequence of keywords. The resulting query vector is compared with repository vectors.

Query stage	Action
User input	Research question/topic entered in the search interface.
Validation	Ensure a usable query is provided.
Embedding	Transform query with the same Sentence Transformer.

Comparison	Calculate vector similarity/distance against stored representations.
Ranking	Order returned papers by relevance.
Presentation	Display titles and available paper information.

6.5 Example Search Behavior

A query can describe a research topic in natural language without reproducing the exact title of a paper. The semantic representation allows the system to retrieve papers whose content is related in meaning. The result should still be treated as a retrieval candidate rather than a guaranteed answer to the user's research question.

6.6 Similarity Calculation

The documented search implementation compares the query embedding with stored embeddings using vector distance. The resulting relationship is converted into a similarity-oriented score and the returned records are sorted so that higher-relevance candidates appear first.

6.7 Ranking Process

33. Generate query embedding.
34. Select repository documents that have valid embeddings.
35. Compute query-to-document vector relationships.
36. Convert the relationship to the application's similarity score.
37. Order candidate documents by similarity.
38. Return a limited set of top results.
39. Display ranked papers in the search interface.

6.8 Interpretation of Scores

A similarity score is useful for ordering results but should not automatically be interpreted as a human-validated relevance percentage. The current project documentation correctly distinguishes successful retrieval execution from quantitative search-quality evaluation. Formal precision, recall and F1 measurements require a labelled query set and relevance judgments.

6.9 Intermediate Semantic Processing

Input	Processing	Observed output
Research PDF	PDF text extraction	Readable research text
Research text	Sentence Transformer	384-dimensional semantic embedding
User query	Same embedding model	Query embedding
Query + stored embeddings	Similarity comparison	Ranked paper results



Figure 4. Semantic-processing evidence included in the original project documentation.

The original report records that the working workflow successfully generated 384-dimensional embeddings and completed semantic search. These observations support the claim that the core semantic pipeline is operational.

6. SEMANTIC SEARCH DESIGN – EVALUATION

6.10 Current Evaluation

The current evaluation is primarily functional. The documented test workflow demonstrates that a query embedding can be generated, similarity search can complete and results can be returned. This is evidence of system execution rather than a benchmark of retrieval quality.

6.11 Recommended Quantitative Evaluation

Metric	Purpose
Precision@K	Measures the proportion of top-K retrieved papers judged relevant.
Recall@K	Measures the proportion of relevant papers retrieved in top-K.
F1-score	Balances precision and recall.
Mean response time	Measures practical query latency.
Top-K relevance rate	Measures the share of relevant results in the returned set.

A labelled evaluation set should contain representative research questions and human relevance judgments. The same query set can then be used to compare semantic search against a conventional keyword baseline.

7. FRONTEND IMPLEMENTATION

7.1 Frontend Design

The frontend provides the user-facing workflow for authentication, uploading, searching and managing research papers. The documented interface was progressively refined to separate repository viewing, paper details and update operations.

7.2 Main Frontend Pages

Page	Purpose
Login	Authenticate a user before protected operations.
Register	Create a new account.
Home / Upload	Select and upload a research-paper PDF.
Search	Enter a natural-language query and view ranked results.
Repository	Display stored papers and management controls.
Paper Details	Open information for one paper.
Update	Upload a replacement PDF for an existing paper.

7.3 Frontend Design Goal

The interface should prioritize clarity: fewer primary controls, consistent navigation, separate pages for detailed actions, responsive layout and readable research-paper cards. The frontend should expose the capabilities of the backend without requiring users to understand the internal embedding process.

7.4 Authentication Interface

The login page provides access to the authenticated application. Registration collects the required account fields and validates them before an account is created. The protected pages are intended to remain inaccessible until a valid session is established.

7.5 Upload Interface

The upload page allows the user to select a PDF and submit it for processing. The backend performs the actual extraction and embedding work. Once successful, the document becomes part of the user's repository.

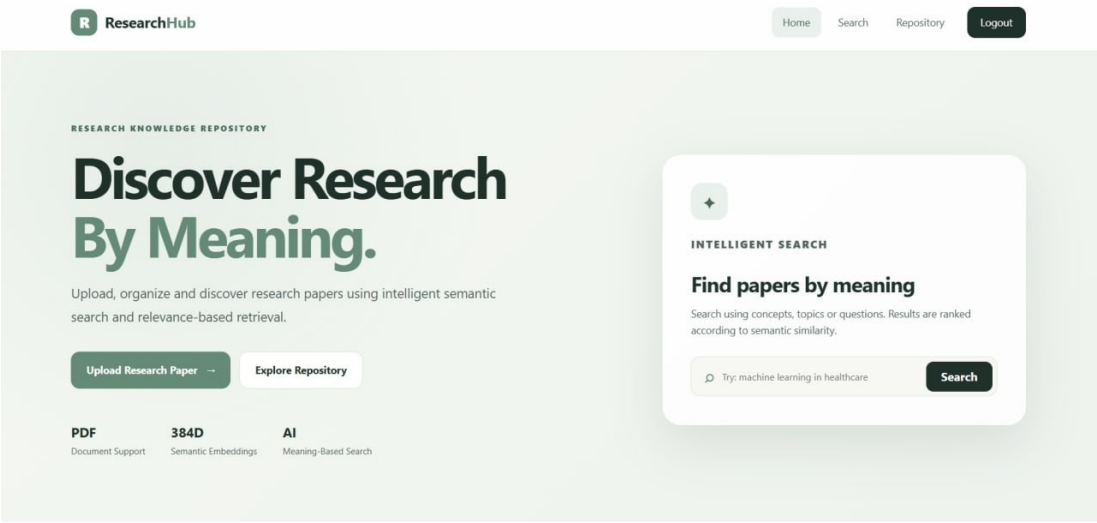


Figure 5. Frontend/home interface evidence from the original documentation.

The screenshot should be replaced or supplemented with the latest running-project screenshot before final submission if the interface has changed.

7. FRONTEND IMPLEMENTATION – REPOSITORY

7.6 Repository Interface

The repository page presents stored research papers and provides access to management operations. A user can open a paper, download it, update it or delete it. In a multi-user implementation, the repository page must display only papers owned by the authenticated account.

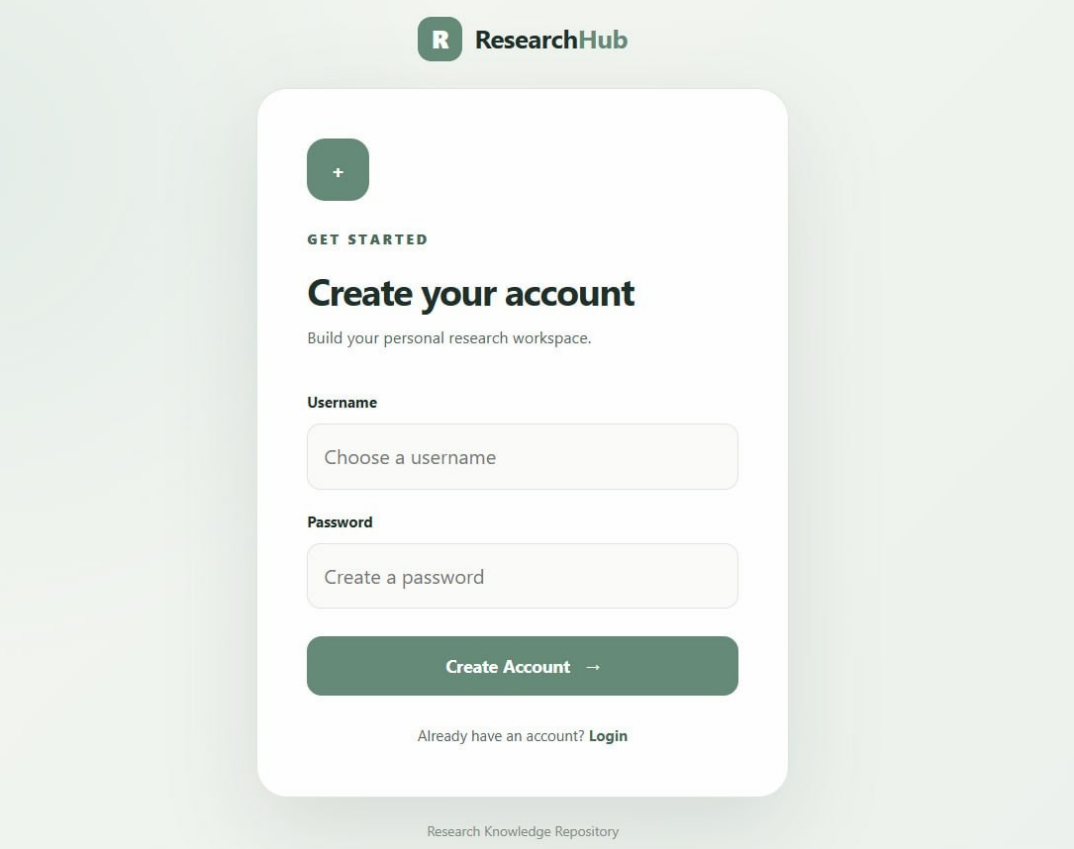


Figure 6. Repository page evidence from the original documentation.

7.7 Repository Information

- Paper title and filename.
- Upload time where available.
- Action controls for view, download, update and delete.
- User-specific filtering for account isolation.
- Navigation back to search and upload functions.

7.8 Search Interface

The search interface allows a user to submit a natural-language research query. The frontend sends the query to the backend, which creates an embedding and performs similarity retrieval. Returned papers are presented as ranked results.

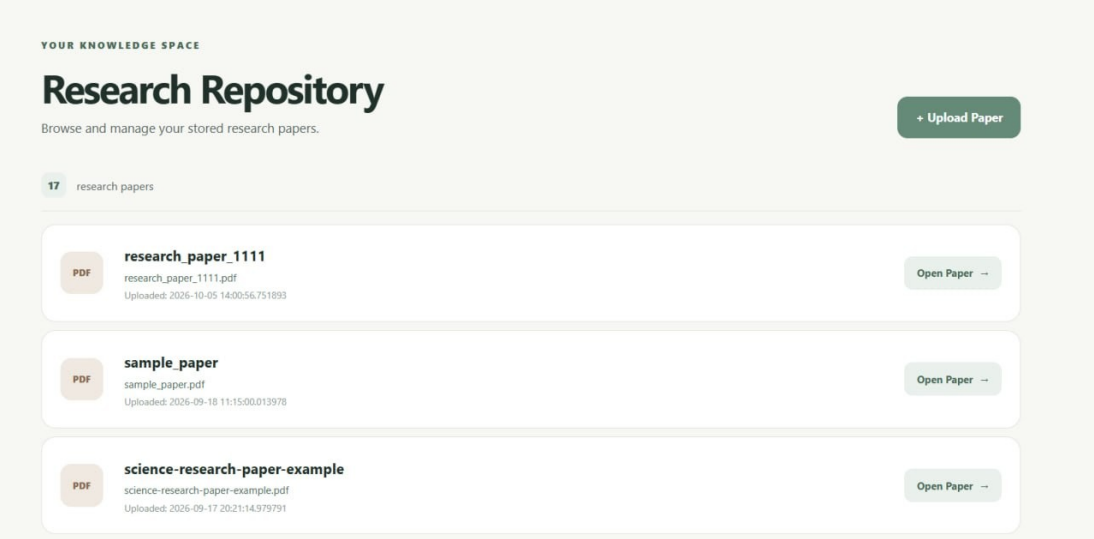


Figure 7. Search/result interface evidence from the original documentation.

7.9 Search User Experience

- Use a clear search input.
- Allow natural-language questions or topic phrases.
- Show the most relevant returned papers first.
- Provide a direct path to paper details.
- Keep search and repository actions visually distinct.

7. FRONTEND IMPLEMENTATION – PAPER MANAGEMENT

7.10 Paper Details

The paper-details page provides a dedicated view for an individual repository record. This separates detailed document information from the repository list and gives the user a focused place to inspect the selected paper.

7.11 Update Interface

The update page accepts a replacement PDF. The backend then extracts new text and regenerates the semantic embedding. This ensures that later searches operate on the current document representation.

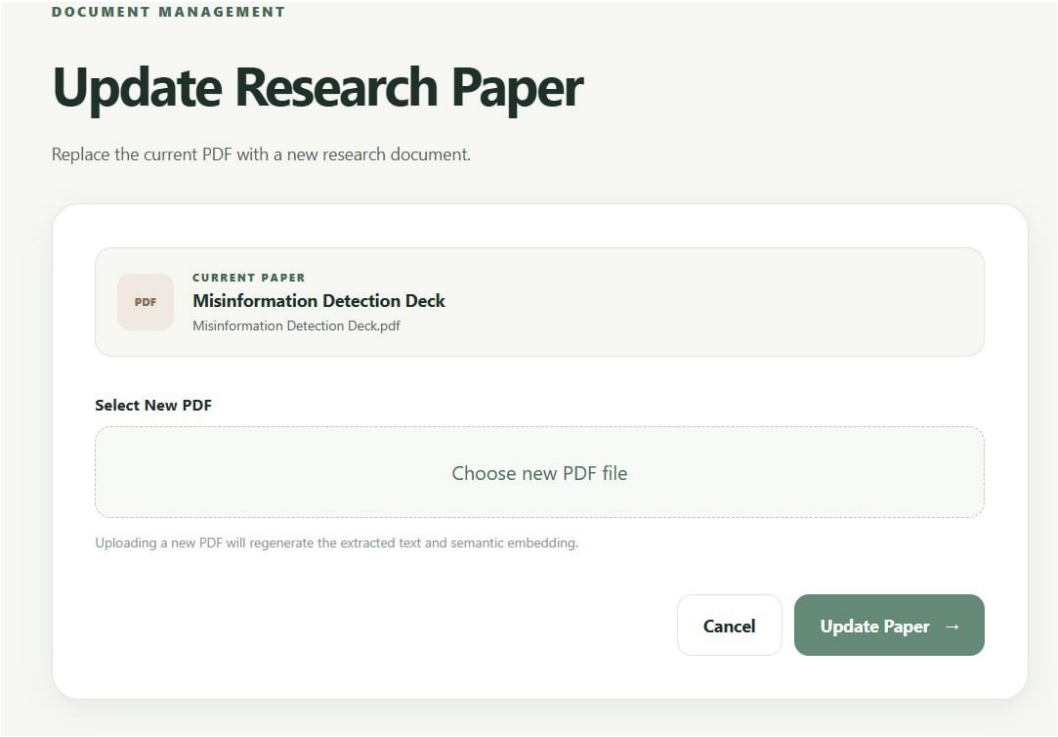


Figure 8. Update or paper-management interface evidence.

7.12 Delete Operation

Deletion removes the selected repository paper. The backend checks that the resource exists before completing the operation. A repeated deletion of

an already removed resource should return a not-found result rather than silently succeeding.

8. BACKEND IMPLEMENTATION

8.1 Flask Application Structure

The backend is implemented using Flask. It coordinates authentication, file handling, PDF extraction, embedding generation, repository storage, semantic search and management operations.

Backend responsibility	Implementation role
Routing	Maps browser requests to application operations.
Authentication	Uses Flask-Login to establish protected sessions.
Password handling	Uses password hashing and verification.
File handling	Validates, saves and serves research PDFs.
PDF processing	Extracts text from uploaded documents.
Embedding	Generates document and query representations.
Search	Compares query and document embeddings.
Repository	Supports listing, viewing, updating and deleting papers.

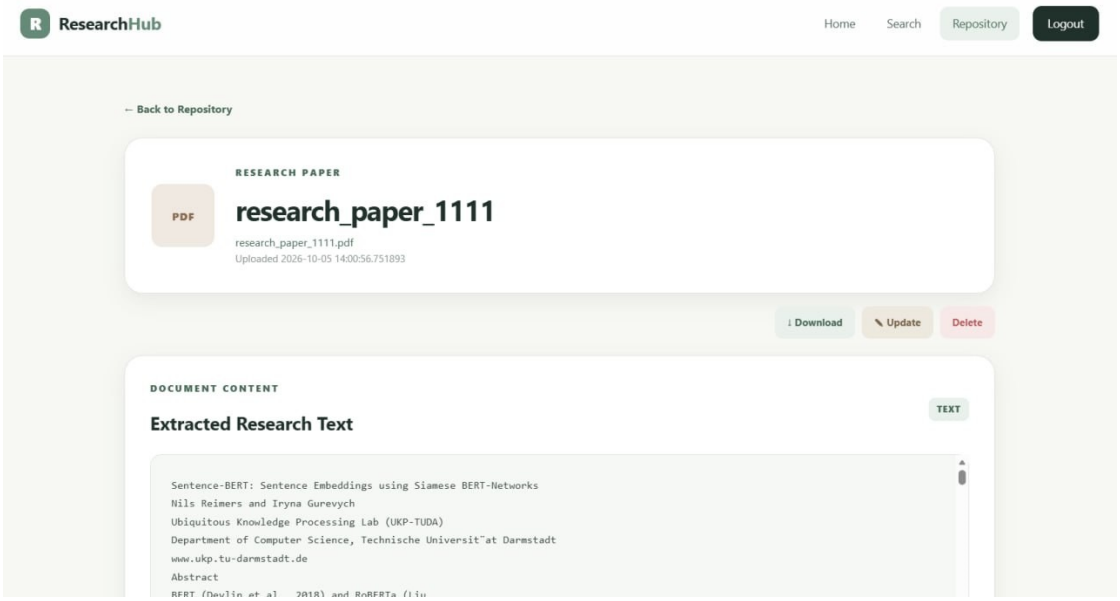


Figure 9. Backend/processing evidence from the original documentation.

8. BACKEND IMPLEMENTATION – PROCESSING SERVICES

8.2 Authentication Processing

Registration validates the username and password, checks whether the username already exists, hashes the password and stores the new user. Login retrieves the account record, verifies the password hash and creates the authenticated user session.

8.3 PDF Processing

The PDF-processing path reads the uploaded document and extracts text from its pages. The extracted content becomes the input to the semantic embedding stage. The original implementation uses a PDF reader for this purpose.

8.4 Embedding Processing

The embedding component loads the Sentence Transformer model and encodes extracted research text. The same model is used when processing a search query. Consistency between these two paths is necessary for meaningful vector comparison.

8.5 Search Processing

The search route embeds the query, compares it with stored vectors and orders the results. The documented implementation returns a limited set of candidates and sorts them by the resulting similarity value.

8. BACKEND IMPLEMENTATION – INTERMEDIATE RESULTS

8.6 Observed Intermediate Results

Operation	Observed result
Application startup	Flask server runs at 127.0.0.1:5000 in the documented test environment.
Authentication	Login completed successfully.
PDF upload	PDF uploaded successfully.
Text extraction	Text extracted successfully.
Embedding	Embedding generated; size 384.
Storage	Paper and embedding stored successfully.
Semantic search	Query embedding and similarity search completed.
Search output	Five results returned in the tested request.
Repository	Repository loaded successfully.
View	Paper details returned HTTP 200 in the tested case.
Download	Paper download returned HTTP 200 in the tested case.
Delete	Paper deletion completed.
Update	New PDF, text and embedding stored successfully.

These observations demonstrate functional execution of the core pipeline. They should not be interpreted as measured search-quality accuracy without a labelled relevance dataset.

9. EXPERIMENTS AND TESTING

9.1 Functional Testing Strategy

Testing focuses on the complete user workflow and the individual repository operations. Each operation is checked against an expected result. Authentication is tested before protected functions; upload is tested before search; and update/delete behavior is checked against repository state.

9.2 Functional Test Cases

Test case	Expected result	Documented status
Valid login	User reaches authenticated application	Passed
Valid PDF upload	PDF processed and stored	Passed
PDF extraction	Readable text produced	Passed
Embedding generation	Semantic vector created	Passed
Natural-language search	Results returned and ranked	Passed
Repository	Stored papers displayed	Passed
View	Paper details page opens	Passed
Download	PDF downloads	Passed
Update	New PDF/text/embedding stored	Passed
Delete	Selected paper removed	Passed

9.3 Testing Principle

A functional pass demonstrates that the application performs the intended operation. It does not by itself establish scalability, retrieval accuracy or production-level security. Those properties require separate evaluation.

9. EXPERIMENTS AND TESTING – ACCOUNT ISOLATION

9.4 Multi-Account Testing

Account isolation is a critical test condition for the repository. A correct multi-user implementation should ensure that documents uploaded by one account do not appear in another account's repository or semantic-search results.

Scenario	Expected behavior
Register new account	New account starts with an empty paper list.
Upload paper as Account A	Paper appears for Account A.
Login as Account B	Account B does not see Account A's paper.
Search as Account B	Account A's paper is excluded from search.
Return to Account A	Account A's paper is visible again.
Update as wrong account	Operation is denied/not found.
Download as wrong account	Operation is denied/not found.
Delete as wrong account	Operation is denied/not found.

9.5 Security Test Interpretation

The key principle is that every paper-related operation should be checked against the authenticated user's ownership. This is a backend/database responsibility, not only a frontend display rule.

10. RESULTS AND DISCUSSION

10.1 Current Results

The project demonstrates a working end-to-end research repository workflow. The documented results confirm successful authentication, PDF upload, text extraction, 384-dimensional embedding generation, semantic-search execution and repository-management operations.

10.2 Practical Contribution

- Combines document storage and intelligent retrieval.
- Transforms PDF content into a reusable semantic representation.
- Supports natural-language search rather than exact keyword dependence alone.
- Connects search results with view/download/update/delete operations.
- Provides a foundation for future intelligent research assistance.

10.3 Evaluation Boundary

The current evidence is primarily functional. The project documentation explicitly recommends a labelled query set and quantitative evaluation before reporting precision, recall or F1 values for retrieval. This distinction keeps the claims aligned with the available evidence.

10. RESULTS AND DISCUSSION – NOVELTY AND ADAPTABILITY

10.4 Novelty

The project does not claim semantic search itself as a new research concept. The practical contribution is the integration of research-paper management, PDF processing, semantic embeddings, natural-language retrieval and repository operations in a single academic prototype.

10.5 Adaptability

Potential environment	Application
University repository	Store and semantically retrieve departmental research papers.
Research laboratory	Organize papers used by a project team.
Student project team	Maintain a shared knowledge collection.
Digital library	Add semantic discovery to document browsing.
Organization	Maintain an internal knowledge repository.

10.6 Future Intelligent Extensions

- Research-paper summarization.
- Question answering over stored papers.
- Recommendation based on research interests.
- Improved text preprocessing and document chunking.
- Hybrid keyword + semantic retrieval.
- Advanced vector indexing for larger repositories.

11. CURRENT STATUS, LIMITATIONS AND FUTURE WORK

11.1 Current Status

- Authentication is working in the documented prototype.
- PDF upload and text extraction are working.
- Embedding generation produces 384-dimensional vectors in the tested workflow.
- Semantic-search requests complete successfully and return ranked results.
- Repository view, paper details, download, update and delete operations have been demonstrated.
- Frontend and backend components are integrated through Flask.

11.2 Limitations

- Quantitative search-quality evaluation is not finalized.
- Current evidence is mainly functional rather than benchmark-based.
- The repository dataset should be expanded for stronger evaluation.
- Ranking quality can be improved through evaluation-driven tuning.
- Advanced summarization, question answering and recommendation are future extensions.

11.3 Future Work

40. Build a larger and more diverse research-paper dataset.
41. Create a labelled query set and measure Precision@K, Recall@K and F1-score.
42. Compare semantic search with conventional keyword search.
43. Improve text preprocessing, chunking and relevance ranking.
44. Strengthen account isolation and ownership checks across every paper operation.
45. Improve frontend usability and search-result presentation.
46. Explore summarization, question answering and recommendation.
47. Improve scalability and deployment readiness.

12. CONCLUSION

The Research Knowledge Repository with Semantic Search progressed from problem identification and proposed methodology to a functioning Flask-based

prototype. The system demonstrates authenticated access, PDF upload, text extraction, semantic embedding generation, storage, natural-language search and repository-management operations. The next stage should focus on quantitative retrieval evaluation, larger datasets, stronger account isolation and additional intelligent research-support capabilities.

REFERENCES

- Sentence Transformers documentation and literature on sentence-level semantic embeddings.
- Literature on BERT and SBERT-based semantic representation.
- FAISS documentation and literature on efficient vector similarity search.
- PDF text extraction documentation.
- Flask documentation for Python web application development.
- Literature reviewed during Review 2 on keyword, content-based and semantic information retrieval.
- Project Review 1, Review 2 and Review 3 materials for the Research Knowledge Repository with Semantic Search.

SCREENSHOT DOCUMENTATION GUIDE

Before final college submission, replace or supplement the embedded evidence with the latest screenshots from the running project. Recommended order: login, registration, home/upload, backend extraction and embedding output, repository, semantic search, paper details, update, download and delete. This keeps the report evidence-oriented in the same manner as the reference Intelligent Email Management documentation.